

Started on	Thursday, 12 June 2025, 1:04 AM
State	Finished
Completed on	Thursday, 12 June 2025, 10:02 AM
Time taken	8 hours 57 mins
Marks	2.00/2.00
Grade	10.00 out of 10.00 (100%)

Viết chương trình đọc vào một đồ thị vô hướng và kiểm tra xem nó có phải là đồ thị phân đôi không.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung của đồ thị.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u, v , cách nhau 1 khoảng trắng, mô tả cung (u, v) .

Đầu ra (Output)

- Nếu đồ thị là đồ thị phân đôi, in ra YES, ngược lại in ra NO.
- Xem thêm các ví dụ bên dưới.

For example:

Input	Result
3 2 1 2 2 3	YES
3 3 1 2 2 3 3 1	NO

Answer: (penalty regime: 10, 20, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4
5  //-----Graph-----
6  typedef struct{
7      int n,m;
8      int A[MAX_SIZE][MAX_SIZE];
9  }Graph;
10
11 void init_graph(Graph* pG, int n){
12     pG->n = n;
13     pG->m = 0;
14     for(int i=0; i<=n;i++){
15         for(int j=0; j<=n;j++){
16             pG->A[i][j] = 0;
17         }
18     }
19 }
20
21 void add_edge(Graph* pG, int u, int v){
22     pG->A[u][v] = 1;
23     if(u!=v){
24         pG->A[v][u] = 1;
25     }
26
27     pG->m++;
28 }
29
30 int adj(Graph* pG, int u, int v){
31     return pG->A[u][v] > 0;
32 }
33
34 //-----DFS-----
35 #define NO_COLOR 0
36 #define RED 1
37 #define BLUE 2
38 #define SUM 3
39
40 int visited[MAX_SIZE];
41 int color[MAX_SIZE];
42 int is_conflict = 0;
43
44 void DFS(Graph* pG, int u, int color_u){
45     // visited[u] = 1;
46     // printf("%d ", u);
47     color[u] = color_u;
48     for(int v=1; v<=pG->n;v++){

```

```

48 for(int v=1; v<=pG->n; v++){
49
50     //xet cac dinh ke cua u
51     if(adj(pG,u,v)){
52         int color_v = color[v];
53         if(color_v==NO_COLOR){
54             DFS(pG, v, SUM - color_u);
55         } else if(color_v==color_u){
56             is_conflict = 1;
57         }
58     }
59 }
60 }
61
62 int main(){
63     Graph G;
64     int n, m;
65     scanf("%d%d", &n,&m);
66
67     init_graph(&G,n);
68
69     for(int i = 0; i < m; i++){
70         int u,v;
71         scanf("%d %d", &u, &v);
72         add_edge(&G, u, v);
73     }
74
75     for(int i = 0; i <= G.n; i++){
76         visited[i] = 0;
77         color[i] = NO_COLOR;
78     }
79
80     DFS(&G,1,BLUE);
81
82     if(is_conflict){
83         printf("NO");
84     }else{
85         printf("YES");
86     }
87
88 }

```

	Input	Expected	Got	
✓	3 2 1 2 2 3	YES	YES	✓
✓	3 3 1 2 2 3 3 1	NO	NO	✓
✓	9 8 1 2 1 3 1 4 1 5 1 6 1 7 1 8 1 9	YES	YES	✓

	Input	Expected	Got	
✓	8 9 1 2 1 3 1 4 1 5 2 7 1 8 5 7 3 6 4 6	YES	YES	✓

Passed all tests! ✓

Question author's solution (C):

```

1 #include <stdio.h>
2
3 /* Khai báo CSDL Graph*/
4 #define MAX_N 100
5 typedef struct {
6     int n, m;
7     int A[MAX_N][MAX_N];
8 } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     if (pG->A[u][v] > 1)
24         printf("da cung (%d, %d)\n", u, v);
25     if (u == v)
26         printf("khuyen %d\n", u);
27
28     pG->m++;
29 }
30
31
32 int adjacent(Graph *pG, int u, int v) {
33     return pG->A[u][v] > 0;
34 }
35
36 #define NO_COLOR 0
37 #define BLUE 1
38 #define RED 2
39
40 //Các biến hỗ trợ
41 int color[MAX_N]; //Lưu trạng thái của các đỉnh
42 int conflict; //Có đụng độ trong khi tô màu không
43
44 //Tô màu các đỉnh của đồ thị bắt đầu từ đỉnh u với màu c
45 void colorize(Graph *pG, int u, int c) {
46     //1. Tô màu c cho u
47     color[u] = c;
48     //2. Xét các đỉnh kề của u
49     for (int v = 1; v <= pG->n; v++)
50         if (adjacent(pG, u, v)) {
51             if (color[v] == NO_COLOR) //2a. Nếu v chưa có màu
52                 colorize(pG, v, 3 - c); //tô màu nó ngược với c
53             else if (color[v] == color[u]) //2b. u và v cùng màu
54                 conflict = 1; //đụng độ, không tô được
55         }
56 }
57
58
59

```

```

60 int main() {
61     //1. Khai báo đồ thị G
62     Graph G;
63     //2. Đọc dữ liệu và dựng đồ thị
64     int n, m, u, v;
65     scanf("%d%d", &n, &m);
66     init_graph(&G, n);
67     for (int e = 0; e < m; e++) {
68         scanf("%d%d", &u, &v);
69         add_edge(&G, u, v);
70     }
71
72     for (int u = 1; u <= G.n; u++)
73         color[u] = NO_COLOR;
74     //2. Khởi tạo biến conflict
75     conflict = 0;
76     //3. Duyệt toàn bộ đồ thị để kiểm tra chu trình
77     for (int u = 1; u <= G.n; u++)
78         if (color[u] == NO_COLOR) //u chưa duyệt
79             colorize(&G, u, BLUE); //gọi DFS(&G, u) để duyệt từ u
80     //4. Kiểm tra conflict
81
82     if (conflict)
83         printf("NO\n");
84     else
85         printf("YES\n");
86
87     return 0;
88 }
89
90

```

Correct

Marks for this submission: 1.00/1.00.

David là huấn luyện viên của một đội bóng gồm **N** thành viên. David muốn chia đội bóng thành hai nhóm. Để tăng tính đa dạng của các thành viên trong nhóm, David quyết định không xếp hai thành viên đã từng thi đấu với nhau vào chung một nhóm. Bạn hãy lập trình giúp David phân chia đội bóng.

Đầu vào (Input):

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên **N** và **M**, tương ứng là số thành viên và số cặp thành viên đã từng thi đấu với nhau.
- M dòng tiếp theo mỗi dòng chứa 2 số nguyên **u v** nói rằng 2 thành viên **u** và **v** đã từng thi đấu chung với nhau.

Đầu ra (Output):

- Nếu phân chia được, hãy in ra các thành viên của mỗi nhóm. Nhóm của thành viên 1 sẽ được in trước, nhóm còn lại in ra sau. Các thành viên trong nhóm được in ra theo thứ tự tăng dần và in trên 1 dòng. Hai thành viên cách nhau 1 khoảng trắng.
- Nếu không thể phân chia, in ra **IMPOSSIBLE**

Xem thêm các ví dụ bên dưới.

For example:

Input	Result
3 2 1 2 2 3	1 3 2
3 3 1 2 2 3 3 1	IMPOSSIBLE
9 8 1 2 1 3 1 4 1 5 1 6 1 7 1 8 1 9	1 2 3 4 5 6 7 8 9

Answer: (penalty regime: 33.3, 66.7, ... %)

```

1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4
5  //-----Graph-----
6  typedef struct{
7      int n,m;
8      int A[MAX_SIZE][MAX_SIZE];
9  }Graph;
10
11 void init_graph(Graph* pG, int n){
12     pG->n = n;
13     pG->m = 0;
14     for(int i=0; i<=n;i++){
15         for(int j=0; j<=n;j++){
16             pG->A[i][j] = 0;
17         }
18     }
19 }
20
21 void add_edge(Graph* pG, int u, int v){
22     pG->A[u][v] = 1;
23     if(u!=v){
24         pG->A[v][u] = 1;
25     }
26     pG->m++;
27 }
28
29
30 int add(Graph* pG, int u, int v){

```

```

30 int adj(Graph* pG, int u, int v){
31     return pG->A[u][v] > 0;
32 }
33
34 //-----Group-----
35 //-----DFS-----
36 #define NO_COLOR 0
37 #define RED 1
38 #define BLUE 2
39 #define SUM 3
40
41 int visited[MAX_SIZE];
42 int color[MAX_SIZE];
43 int is_conflict = 0;
44
45 void DFS(Graph* pG, int u, int color_u){
46     // visited[u] = 1;
47     // printf("%d ", u);
48     color[u] = color_u;
49     for(int v=1; v<=pG->n; v++){
50
51         //xet cac dinh ke cua u
52         if(adj(pG,u,v)){
53             int color_v = color[v];
54             if(color_v==NO_COLOR){
55                 DFS(pG, v, SUM - color_u);
56             } else if(color_v==color_u){
57                 is_conflict = 1;
58             }
59         }
60     }
61 }
62
63 int main(){
64     Graph G;
65     int n, m;
66     scanf("%d%d", &n,&m);
67
68     init_graph(&G,n);
69
70     for(int i = 0; i < m; i++){
71         int u,v;
72         scanf("%d %d", &u, &v);
73         add_edge(&G, u, v);
74     }
75
76     for(int i = 0; i <= G.n; i++){
77         visited[i] = 0;
78         color[i] = NO_COLOR;
79     }
80
81     DFS(&G,1,BLUE);
82
83     if(is_conflict){
84         printf("IMPOSSIBLE");
85     }else{
86         for(int i = 1; i<=G.n; i++){
87             if(color[i] == BLUE){
88                 printf("%d ", i);
89             }
90         }
91         printf("\n");
92         for(int i = 1; i<=G.n; i++){
93             if(color[i] == RED){
94                 printf("%d ", i);
95             }
96         }
97     }
98 }
99

```

Debug: source code from all test runs

Run 1


```

#include <stdio.h>

#define MAX_SIZE 100

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

void init_graph(Graph* pG, int n){
    pG->n = n;
    pG->m = 0;
    for(int i=0; i<=n;i++){
        for(int j=0; j<=n;j++){
            pG->A[i][j] = 0;
        }
    }
}

void add_edge(Graph* pG, int u, int v){
    pG->A[u][v] = 1;
    if(u!=v){
        pG->A[v][u] = 1;
    }

    pG->m++;
}

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

//-----Group-----
//-----DFS-----
#define NO_COLOR 0
#define RED 1
#define BLUE 2
#define SUM 3

int visited[MAX_SIZE];
int color[MAX_SIZE];
int is_conflict = 0;

void DFS(Graph* pG, int u, int color_u){
    //    visited[u] = 1;
    //    printf("%d ", u);
    color[u] = color_u;
    for(int v=1; v<=pG->n; v++){

        //xet cac dinh ke cua u
        if(adj(pG,u,v)){
            int color_v = color[v];
            if(color_v==NO_COLOR){
                DFS(pG, v, SUM - color_u);
            } else if(color_v==color_u){
                is_conflict = 1;
            }
        }
    }
}

int main(){
    Graph G;
    int n, m;
    scanf("%d%d", &n,&m);

    init_graph(&G,n);

    for(int i = 0; i < m; i++){
        int u,v;

```

```
        scanf("%d %d", &u, &v);
        add_edge(&G, u, v);
    }

    for(int i = 0; i <= G.n; i++){
        visited[i] = 0;
        color[i] = NO_COLOR;
    }

    DFS(&G,1,BLUE);

    if(is_conflict){
        printf("IMPOSSIBLE");
    }else{
        for(int i = 1; i<=G.n; i++){
            if(color[i] == BLUE){
                printf("%d ", i);
            }
        }
        printf("\n");
        for(int i = 1; i<=G.n; i++){
            if(color[i] == RED){
                printf("%d ", i);
            }
        }
    }
}
```

Run 2

```

#include <stdio.h>

#define MAX_SIZE 100

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

void init_graph(Graph* pG, int n){
    pG->n = n;
    pG->m = 0;
    for(int i=0; i<=n;i++){
        for(int j=0; j<=n;j++){
            pG->A[i][j] = 0;
        }
    }
}

void add_edge(Graph* pG, int u, int v){
    pG->A[u][v] = 1;
    if(u!=v){
        pG->A[v][u] = 1;
    }

    pG->m++;
}

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

//-----Group-----
//-----DFS-----
#define NO_COLOR 0
#define RED 1
#define BLUE 2
#define SUM 3

int visited[MAX_SIZE];
int color[MAX_SIZE];
int is_conflict = 0;

void DFS(Graph* pG, int u, int color_u){
    //    visited[u] = 1;
    //    printf("%d ", u);
    color[u] = color_u;
    for(int v=1; v<=pG->n; v++){

        //xet cac dinh ke cua u
        if(adj(pG,u,v)){
            int color_v = color[v];
            if(color_v==NO_COLOR){
                DFS(pG, v, SUM - color_u);
            } else if(color_v==color_u){
                is_conflict = 1;
            }
        }
    }
}

int main(){
    Graph G;
    int n, m;
    scanf("%d%d", &n,&m);

    init_graph(&G,n);

    for(int i = 0; i < m; i++){
        int u,v;

```

```

        scanf("%d %d", &u, &v);
        add_edge(&G, u, v);
    }

    for(int i = 0; i <= G.n; i++){
        visited[i] = 0;
        color[i] = NO_COLOR;
    }

    DFS(&G,1,BLUE);

    if(is_conflict){
        printf("IMPOSSIBLE");
    }else{
        for(int i = 1; i<=G.n; i++){
            if(color[i] == BLUE){
                printf("%d ", i);
            }
        }
        printf("\n");
        for(int i = 1; i<=G.n; i++){
            if(color[i] == RED){
                printf("%d ", i);
            }
        }
    }
}

```

Run 3

```

#include <stdio.h>

#define MAX_SIZE 100

//-----Graph-----
typedef struct{
    int n,m;
    int A[MAX_SIZE][MAX_SIZE];
}Graph;

void init_graph(Graph* pG, int n){
    pG->n = n;
    pG->m = 0;
    for(int i=0; i<=n;i++){
        for(int j=0; j<=n;j++){
            pG->A[i][j] = 0;
        }
    }
}

void add_edge(Graph* pG, int u, int v){
    pG->A[u][v] = 1;
    if(u!=v){
        pG->A[v][u] = 1;
    }

    pG->m++;
}

int adj(Graph* pG, int u, int v){
    return pG->A[u][v] > 0;
}

//-----Group-----
//-----DFS-----
#define NO_COLOR 0
#define RED 1
#define BLUE 2
#define SUM 3

int visited[MAX_SIZE];
int color[MAX_SIZE];
int is_conflict = 0;

void DFS(Graph* pG, int u, int color_u){
    //    visited[u] = 1;
    //    printf("%d ", u);
    color[u] = color_u;
    for(int v=1; v<=pG->n; v++){

        //xet cac dinh ke cua u
        if(adj(pG,u,v)){
            int color_v = color[v];
            if(color_v==NO_COLOR){
                DFS(pG, v, SUM - color_u);
            } else if(color_v==color_u){
                is_conflict = 1;
            }
        }
    }
}

int main(){
    Graph G;
    int n, m;
    scanf("%d%d", &n,&m);

    init_graph(&G,n);

    for(int i = 0; i < m; i++){
        int u,v;

```

```
scanf("%d %d", &u, &v);
add_edge(&G, u, v);
}

for(int i = 0; i <= G.n; i++){
    visited[i] = 0;
    color[i] = NO_COLOR;
}

DFS(&G,1,BLUE);

if(is_conflict){
    printf("IMPOSSIBLE");
}else{
    for(int i = 1; i<=G.n; i++){
        if(color[i] == BLUE){
            printf("%d ", i);
        }
    }
    printf("\n");
    for(int i = 1; i<=G.n; i++){
        if(color[i] == RED){
            printf("%d ", i);
        }
    }
}
}
```

	Input	Expected	Got	
✓	3 2 1 2 2 3	1 3 2	1 3 2	✓
✓	3 3 1 2 2 3 3 1	IMPOSSIBLE	IMPOSSIBLE	✓
✓	9 8 1 2 1 3 1 4 1 5 1 6 1 7 1 8 1 9	1 2 3 4 5 6 7 8 9	1 2 3 4 5 6 7 8 9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.